



K.R. Mangalam University
School of Engineering & Technology

Fundamentals Of Java Programming Lab
(ENCA203) Assignment 5
Student Management System

Submitted by:

Name: Shivam Gaur

Roll no. 2401201016

Class: BCA (h) (AI & DS)

Submitted to:

Dr. Manish Kumar

GitHub Repository:

https://github.com/shivamgaur1255-sketch/java_assignments-

Codes:

```
J Calculator.java 1 StudentRecordSystem.java X
J StudentRecordSystem.java > StudentRecordSystem > addStudent()
1 import java.io.*;
2 import java.util.*;
3 import java.util.concurrent.atomic.AtomicBoolean;
4 import java.util.regex.Pattern;
5
6 abstract class Person {
7     protected String name;
8     protected String email;
9     public Person(String name, String email) {
10         this.name = name;
11         this.email = email;
12     }
13     public abstract void displayInfo();
14 }
15
16 class Student extends Person {
17     private int rollNo;
18     private String course;
19     private double marks;
20     private String grade;
21     public Student(int rollNo, String name, String email, String course, double marks) {
22         super(name, email);
23         this.rollNo = rollNo;
24         this.course = course;
25         this.marks = marks;
26         this.grade = calculateGrade();
27     }
28     public int getRollNo() { return rollNo; }
29     public String getName() { return name; }
30     public String getEmail() { return email; }
31     public String getCourse() { return course; }
32     public double getMarks() { return marks; }
33     public String getGrade() { return grade; }
34     public void setName(String name) { this.name = name; }
35     public void setEmail(String email) { this.email = email; }
36     public void setCourse(String course) { this.course = course; }
37     public void setMarks(double marks) { this.marks = marks; this.grade = calculateGrade(); }
38     private String calculateGrade() {
39         if (marks >= 90) return "A";
40         if (marks >= 75) return "B";
41         if (marks >= 60) return "C";
42     }
43 }
44
45 public void displayDetails() {
46     System.out.println("Roll No: " + rollNo);
47     System.out.println("Name: " + name);
48     System.out.println("Email: " + email);
49     System.out.println("Course: " + course);
50     System.out.println("Marks: " + marks);
51     System.out.println("Grade: " + grade);
52     System.out.println();
53 }
54
55 @Override
56 public void displayInfo() { displayDetails(); }
57 public String toFileString() {
58     return rollNo + "|" + escape(name) + "|" + escape(email) + "|" + escape(course) + "|" + marks;
59 }
60 private String escape(String s) {
61     return s.replace(target:"|", replacement:"\\|");
62 }
63 public static String unescape(String s) {
64     return s.replace(target:"\\|", replacement:"|");
65 }
66 }
67
68 interface RecordActions {
69     void addStudent() throws Exception;
70     void deleteStudent() throws StudentNotFoundException;
71     void updateStudent() throws StudentNotFoundException;
72     List<Student> searchStudentByName(String name);
73     void viewAllStudents();
74     void sortByMarksDescending();
75     void saveToFile() throws IOException;
76 }
77
78 class StudentNotFoundException extends Exception {
79     public StudentNotFoundException(String msg) { super(msg); }
80 }
```

```
Explorer (Ctrl+Shift+E) StudentRecordSystem.java X
StudentRecordSystem.java > StudentRecordSystem > addStudent()
77 class StudentNotFoundException extends Exception {
78     public StudentNotFoundException(String msg) { super(msg); }
79 }
80
81 class Loader implements Runnable {
82     private AtomicBoolean running;
83     private String task;
84     public Loader(AtomicBoolean running, String task) {
85         this.running = running;
86         this.task = task;
87     }
88     @Override
89     public void run() {
90         try {
91             System.out.print(task);
92             int dots = 0;
93             while (running.get()) {
94                 Thread.sleep(millis:400);
95                 System.out.print(s:".");
96                 dots++;
97                 if (dots % 10 == 0) {
98                     System.out.print("\r" + task);
99                 }
100             }
101             System.out.println();
102         } catch (InterruptedException e) {
103         }
104     }
105 }
106
107 public class StudentRecordSystem implements RecordActions {
108     private Map<Integer, Student> studentMap = new HashMap<>();
109     private Scanner sc = new Scanner(System.in);
110     private final String fileName = "students.txt";
111     public StudentRecordSystem() {
112         try {
113             loadFromFile();
114             if (!studentMap.isEmpty()) {
115                 System.out.println(x:"Loaded students:");
116                 viewAllStudents();
117             } else {

```

```
J Calculator.java J StudentRecordSystem.java X
StudentRecordSystem.java > StudentRecordSystem > addStudent()
107 public class StudentRecordSystem implements RecordActions {
111     public StudentRecordSystem() {
116         viewAllStudents();
117     } else {
118         System.out.println(x:"No students loaded from file.");
119     }
120 } catch (Exception e) {
121     System.out.println("Error loading students: " + e.getMessage());
122 }
123
124 private void loadFromFile() throws IOException {
125     File f = new File(fileName);
126     if (!f.exists()) {
127         f.createNewFile();
128         return;
129     }
130     try (BufferedReader br = new BufferedReader(new FileReader(f))) {
131         String line;
132         while ((line = br.readLine()) != null) {
133             if (line.trim().isEmpty()) continue;
134             String[] parts = splitEscaped(line, sep: '|');
135             if (parts.length < 5) continue;
136             try {
137                 int roll = Integer.parseInt(parts[0]);
138                 String name = Student.unescape(parts[1]);
139                 String email = Student.unescape(parts[2]);
140                 String course = Student.unescape(parts[3]);
141                 double marks = Double.parseDouble(parts[4]);
142                 Student s = new Student(roll, name, email, course, marks);
143                 studentMap.put(roll, s);
144             } catch (NumberFormatException ignored) {}
145         }
146     }
147 }
148 private static String[] splitEscaped(String s, char sep) {
149     List<String> out = new ArrayList<>();
150     StringBuilder cur = new StringBuilder();
151     boolean esc = false;
152     for (int i = 0; i < s.length(); i++) {
153         char c = s.charAt(i);
154         if (esc) {

```

```
J StudentRecordSystem.java > StudentRecordSystem > addStudent()
187 public class StudentRecordSystem implements RecordActions {
188     private static String[] splitEscaped(String s, char sep) {
189         char c = s.charAt(1);
190         if (esc) {
191             cur.append(c);
192             esc = false;
193         } else {
194             if (c == '\\') {
195                 esc = true;
196             } else if (c == sep) {
197                 out.add(cur.toString());
198                 cur.setLength(newLength:0);
199             } else {
200                 cur.append(c);
201             }
202         }
203         out.add(cur.toString());
204         return out.toArray(new String[0]);
205     }
206 }
207 @Override
208 public void addStudent() throws Exception {
209     int roll = readInt(prompt:"Enter Roll No: ");
210     if (studentMap.containsKey(roll)) {
211         System.out.println(x:"Roll number already exists. Add aborted.");
212         return;
213     }
214     String name = readNonEmptyString(prompt:"Enter Name: ");
215     String email = readValidatedEmail(prompt:"Enter Email: ");
216     String course = readNonEmptyString(prompt:"Enter Course: ");
217     double marks = readDoubleInRange(prompt:"Enter Marks: ", min:0, max:100);
218     AtomicBoolean running = new AtomicBoolean(initialValue:true);
219     Thread loader = new Thread(new Loader(running, task:"Adding student"));
220     loader.start();
221     Thread.sleep(1000);
222     Student s = new Student(roll, name, email, course, marks);
223     studentMap.put(roll, s);
224     running.set(newValue:false);
225     loader.join();
226     System.out.println(x:"Student added.");
227 }
228 @Override
229 public void deleteStudent() throws StudentNotFoundException {
230     String name = readNonEmptyString(prompt:"Enter name to delete: ");
231     List<Integer> toDelete = new ArrayList<>();
232     for (Student s : studentMap.values()) {
233         if (s.getName().equalsIgnoreCase(name)) toDelete.add(s.getRollNo());
234     }
235     if (toDelete.isEmpty()) throw new StudentNotFoundException(msg:"Student not found.");
236     for (int r : toDelete) studentMap.remove(r);
237     System.out.println(x:"Student record deleted.");
238 }
239 @Override
240 public void updateStudent() throws StudentNotFoundException {
241     int roll = readInt(prompt:"Enter Roll No to update: ");
242     Student s = studentMap.get(roll);
243     if (s == null) throw new StudentNotFoundException(msg:"Student not found.");
244     System.out.println(x:"Leave field empty to keep current value.");
245     System.out.println("Current Name: " + s.getName());
246     String name = readStringAllowEmpty(prompt:"Enter New Name: ");
247     if (!name.isEmpty()) s.setName(name);
248     System.out.println("Current Email: " + s.getEmail());
249     String email = readStringAllowEmpty(prompt:"Enter New Email: ");
250     if (!email.isEmpty()) {
251         if (!isValidEmail(email)) System.out.println(x:"Invalid email format. Keeping old email.");
252         else s.setEmail(email);
253     }
254     System.out.println("Current Course: " + s.getCourse());
255     String course = readStringAllowEmpty(prompt:"Enter New Course: ");
256     if (!course.isEmpty()) s.setCourse(course);
257     System.out.println("Current Marks: " + s.getMarks());
258     String marksStr = readStringAllowEmpty(prompt:"Enter New Marks: ");
259     if (!marksStr.isEmpty()) {
260         try {
261             double m = Double.parseDouble(marksStr);
262             if (m < 0 || m > 100) System.out.println(x:"Marks out of range. Keeping old marks.");
263             else s.setMarks(m);
264         } catch (NumberFormatException e) {
265             System.out.println(x:"Invalid marks format. Keeping old marks.");
266         }
267     }
268 }
```

```
J Calculator.java 1 J StudentRecordSystem.java X StudentApp
J StudentRecordSystem.java > StudentRecordSystem > addStudent()
107 public class StudentRecordSystem implements RecordActions {
204 public void updateStudent() throws StudentNotFoundException {
225     double m = Double.parseDouble(marksStr);
226     if (m < 0 || m > 100) System.out.println(x:"Marks out of range. Keeping old marks.");
227     else s.setMarks(m);
228     } catch (NumberFormatException e) {
229     } System.out.println(x:"Invalid marks. Keeping old marks.");
230     }
231     }
232     System.out.println(x:"Student updated.");
233     }
234     @Override
235     public List<Student> searchStudentByName(String name) {
236     List<Student> res = new ArrayList<>();
237     for (Student s : studentMap.values()) {
238     if (s.getName().equalsIgnoreCase(name)) res.add(s);
239     }
240     return res;
241     }
242     @Override
243     public void viewAllStudents() {
244     if (studentMap.isEmpty()) {
245     System.out.println(x:"No students available.");
246     return;
247     }
248     Iterator<Student> it = studentMap.values().iterator();
249     while (it.hasNext()) {
250     Student s = it.next();
251     s.displayDetails();
252     }
253     }
254     @Override
255     public void sortByMarksDescending() {
256     List<Student> list = new ArrayList<>(studentMap.values());
257     list.sort(new Comparator<Student>() {
258     @Override
259     public int compare(Student o1, Student o2) {
260     return Double.compare(o2.getMarks(), o1.getMarks());
261     }
262     });
263     if (list.isEmpty()) {
```

```
J Calculator.java 1 J StudentRecordSystem.java X StudentApp
J StudentRecordSystem.java > StudentRecordSystem > addStudent()
107 public class StudentRecordSystem implements RecordActions {
255 public void sortByMarksDescending() {
257     list.sort(new Comparator<Student>() {
261     });
262     });
263     if (list.isEmpty()) {
264     System.out.println(x:"No students to sort.");
265     return;
266     }
267     System.out.println(x:"Sorted Student List by Marks:");
268     Iterator<Student> it = list.iterator();
269     while (it.hasNext()) {
270     it.next().displayDetails();
271     }
272     }
273     @Override
274     public void saveToFile() throws IOException {
275     AtomicBoolean running = new AtomicBoolean(initialValue:true);
276     Thread loader = new Thread(new Loader(running, task:"Saving records");
277     loader.start();
278     try (BufferedWriter bw = new BufferedWriter(new FileWriter(fileName, append:false))) {
279     for (Student s : studentMap.values()) {
280     bw.write(s.toString());
281     bw.newLine();
282     }
283     } finally {
284     running.set(newValue:false);
285     try { loader.join(); } catch (InterruptedException ignored) {}
286     }
287     System.out.println(x:"Saved and exiting.");
288     }
289     private int readInt(String prompt) {
290     while (true) {
291     try {
292     System.out.print(prompt);
293     String line = sc.nextLine().trim();
294     return Integer.parseInt(line);
295     } catch (NumberFormatException e) {
296     System.out.println(x:"Invalid integer. Try again.");
297     }
```

```
J Calculator.java 1 J StudentRecordSystem.java X
J StudentRecordSystem.java > StudentRecordSystem > addStudent()
107 public class StudentRecordSystem implements RecordActions {
289 private int readInt(String prompt) {
290     // ...
291     // ...
292     // ...
293     // ...
294     // ...
295     } catch (NumberFormatException e) {
296         System.out.println(x:"Invalid integer. Try again.");
297     }
298 }
299
300 private double readDoubleInRange(String prompt, double min, double max) {
301     while (true) {
302         try {
303             System.out.print(prompt);
304             String line = sc.nextLine().trim();
305             double d = Double.parseDouble(line);
306             if (d < min || d > max) {
307                 System.out.println("Value must be between " + min + " and " + max + ".");
308                 continue;
309             }
310             return d;
311         } catch (NumberFormatException e) {
312             System.out.println(x:"Invalid number. Try again.");
313         }
314     }
315 }
316 private String readNonEmptyString(String prompt) {
317     while (true) {
318         System.out.print(prompt);
319         String s = sc.nextLine().trim();
320         if (s.isEmpty()) System.out.println(x:"Field cannot be empty.");
321         else return s;
322     }
323 }
324 private String readStringAllowEmpty(String prompt) {
325     System.out.print(prompt);
326     return sc.nextLine().trim();
327 }
328 private String readValidatedEmail(String prompt) {
329     while (true) {
330         System.out.print(prompt);
331         String e = sc.nextLine().trim();
332         if (e.isEmpty()) {
333             System.out.println(x:"Email cannot be empty.");
334             continue;
335         }
336         if (!isValidEmail(e)) {
337             System.out.println(x:"Invalid email format.");
338             continue;
339         }
340         return e;
341     }
342 }
343
344 private boolean isValidEmail(String email) {
345     String regex = "^[\\w.-]+@[\\w.-]+\\.[A-Za-z]{2,}$";
346     return Pattern.matches(regex, email);
347 }
348 private void menu() {
349     while (true) {
350         System.out.println(x:"----- Capstone Student Menu -----");
351         System.out.println(x:"1. Add Student");
352         System.out.println(x:"2. View All Students");
353         System.out.println(x:"3. Search by Name");
354         System.out.println(x:"4. Delete by Name");
355         System.out.println(x:"5. Sort by Marks");
356         System.out.println(x:"6. Update by Roll No");
357         System.out.println(x:"7. Save and Exit");
358         System.out.print(s:"Enter choice: ");
359         String choice = sc.nextLine().trim();
360         try {
361             switch (choice) {
362                 case "1": addStudent(); break;
363                 case "2": viewAllStudents(); break;
364                 case "3":
365                     String nm = readNonEmptyString(prompt:"Enter name to search: ");
366                     List<Student> found = searchStudentByName(nm);
367                     if (found.isEmpty()) System.out.println(x:"No student found.");
368                     else {
```

```
J Calculator.java 1 J StudentRecordSystem.java • StudentApp
J StudentRecordSystem.java > StudentRecordSystem > menu()
107 public class StudentRecordSystem implements RecordActions {
348 private void menu() {
362     case "1": addStudent(); break;
363     case "2": viewAllStudents(); break;
364     case "3":
365         String nm = readNonEmptyString(prompt: "Enter name to search: ");
366         List<Student> found = searchStudentByName(nm);
367         if (found.isEmpty()) System.out.println("No student found.");
368         else {
369             System.out.println(x: "Student Info:");
370             for (Student s : found) s.displayDetails();
371         }
372         break;
373     case "4":
374         try {
375             deleteStudent();
376         } catch (StudentNotFoundException e) {
377             System.out.println(e.getMessage());
378         }
379         break;
380     case "5": sortByMarksDescending(); break;
381     case "6":
382         try {
383             updateStudent();
384         } catch (StudentNotFoundException e) {
385             System.out.println(e.getMessage());
386         }
387         break;
388     case "7":
389         try {
390             saveToFile();
391         } catch (IOException e) {
392             System.out.println("Error saving: " + e.getMessage());
393         }
394         return;
395     default:
396         System.out.println(x: "Invalid choice.");
397 }
398 } catch (Exception e) {
399     System.out.println("Error: " + e.getMessage());
400 }
}

Run | Debug
403 public static void main(String[] args) {
404     StudentRecordSystem app = new StudentRecordSystem();
405     app.menu();
406 }
407
408
```

Output:

```
PS C:\Users\Shivam Gaur\OneDrive\ドキュメント\java coding> & 'C:\Program Files\Java\jdk-21\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Shivam Gaur\AppData\Roaming\Code\User\workspaceStorage\b86f09e45bbf9afd5a31de64f223dfa7\redhat.java\jdt_ws\java coding_d70f91f4\bin' 'StudentRecordSystem'
No students loaded from file.
===== Capstone Student Menu =====
1. Add Student
2. View All Students
3. Search by Name
4. Delete by Name
5. Sort by Marks
6. Update by Roll No
7. Save and Exit
Enter choice: 1
Enter Roll No: Ram
Invalid integer. Try again.
Enter Roll No: 101
Enter Name: Ram
Enter Email: ram@mail.com
Enter Course: Bca
Enter Marks: 83
Adding student..
Student added.
===== Capstone Student Menu =====
1. Add Student
2. View All Students
3. Search by Name
4. Delete by Name
5. Sort by Marks
6. Update by Roll No
7. Save and Exit
Enter choice: 1
Enter Roll No: 102
Enter Name: Shyam
Enter Email: shyam@mail.com
Enter Course: B.Tech
Enter Marks: 79
Adding student..
Student added.
===== Capstone Student Menu =====
1. Add Student
2. View All Students
3. Search by Name
4. Delete by Name
5. Sort by Marks
6. Update by Roll No
7. Save and Exit
Enter choice:
Invalid choice.
===== Capstone Student Menu =====
1. Add Student
2. View All Students
===== Capstone Student Menu =====
1. Add Student
2. View All Students
3. Search by Name
4. Delete by Name
5. Sort by Marks
6. Update by Roll No
7. Save and Exit
Enter choice: 2
Roll No: 101
Name: Ram
Email: ram@mail.com
Course: Bca
Marks: 83.0
Grade: B

Roll No: 102
Name: Shyam
Email: shyam@mail.com
Course: B.Tech
Marks: 79.0
Grade: B

===== Capstone Student Menu =====
1. Add Student
2. View All Students
3. Search by Name
4. Delete by Name
5. Sort by Marks
6. Update by Roll No
7. Save and Exit
Enter choice: 3
Enter name to search: Ram
Student Info:
Roll No: 101
Name: Ram
Email: ram@mail.com
Course: Bca
Marks: 83.0
Grade: B

===== Capstone Student Menu =====
1. Add Student
2. View All Students
3. Search by Name
4. Delete by Name
5. Sort by Marks
6. Update by Roll No
7. Save and Exit
Enter choice: 4
```


PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS

StudentApp

1

🔍

🔗

🔧

🔧

🔧

4. Delete by Name

5. Sort by Marks

6. Update by Roll No

7. Save and Exit

Enter choice: 7

Saving records.

2. View All Students

3. Search by Name

4. Delete by Name

5. Sort by Marks

6. Update by Roll No

7. Save and Exit

Enter choice: 7

Saving records.

3. Search by Name

4. Delete by Name

5. Sort by Marks

6. Update by Roll No

7. Save and Exit

Enter choice: 7

Saving records.

6. Update by Roll No

7. Save and Exit

Enter choice: 7

Saving records.

Saved and exiting.

PS C:\Users\Shivam Gaur\OneDrive\ドキュメント\java coding>